

Projet Optimisation Finances

Javier Peña Colada, Martin Leivasion Fiscale, et Autiste Pras

1 Modélisation, pseudo code et résolution “à la main”

1. Soit $G = (X, U)$ un graphe orienté, où E = Euros, D = Dollars, F = Francs Suisses, J = Yens, on a:

$$X = \{E, D, F, J\}$$

$$U = \{(E, D), (D, E), (J, D), (D, J), (F, D), (D, F), (E, F), (F, E), (E, J), (J, E), (J, F), (F, J)\}$$

$$\begin{aligned} v(E, D) &= 1,19 & v(D, E) &= 0,84 \\ v(D, J) &= 1,12 & v(J, D) &= 0,89 \\ v(D, F) &= 1,37 & v(F, D) &= 0,73 \\ v(E, F) &= 1,62 & v(F, E) &= 0,62 \\ v(E, J) &= 1,33 & v(J, E) &= 0,75 \\ v(J, F) &= 1,22 & v(F, J) &= 0,82 \end{aligned}$$

2.

$$v(\mu) = \prod_{i=0}^{n-1} v(x_i, x_{i+1})$$

où x_i, x_{i+1} sont les sommets du graphe et où $v(x_i, x_{i+1})$ désigne la valeur de l'arête reliant x_i à x_{i+1} (c'est-à-dire le taux de change entre ces deux monnaies). Le produit des $v(x_i, x_{i+1})$ représente les échanges successifs réalisés.

3. En partant d'un montant en euros, on cherche à effectuer des échanges successifs vers d'autres monnaies afin d'augmenter le montant final obtenu en euros.

On cherche donc à trouver un cycle $\mu = (x_0, x_1, \dots, x_n)$, où $x_0 = x_n = E$ et tel que $v(\mu)$ soit maximal, c'est à dire tel que $v(\mu) = \prod_{i=0}^{n-1} v(x_i, x_{i+1})$ soit le plus grand possible.

Si $v(\mu) > 1$, le cycle est profitable : plus $v(\mu)$ est grand, plus le profit est important.

Si $v(\mu) < 1$, le cycle est défavorable : plus $v(\mu)$ est petit, plus la perte est importante.

Si $v(\mu) = 1$, le cycle est neutre, sans gain ni perte.

4. Si le graphe G contient un cycle fermé μ tel que $v(\mu) > 1$, alors en répétant ce cycle une infinité de fois, on peut espérer un profit infini.

5. Voir les pseudo-codes sur la page 2.

6. La meilleure séquence d'échange est **{E, D, F, E}** et offre un taux multiplicatif de **1.010786**. C'est-à-dire qu'en investissant 100€, on aurait après ces 3 échanges 101,0786€.

Listing 1: Dictionnaire d'adjacence de G

```
G = {
  'E': {'D': 1.19, 'J': 1.33, 'F': 1.62},
  'D': {'E': 0.84, 'J': 1.12, 'F': 1.37},
  'J': {'E': 0.75, 'D': 0.89, 'F': 1.22},
  'F': {'E': 0.62, 'D': 0.73, 'J': 0.82}
}
```

Algorithm 1 meilleur_cycle($G, debut, p$), temps: $O(|X|^p)$, mémoire: $O(p)$

```

1: Entrées : dictionnaire d'adjacence  $G$  des taux  $G[i, j]$ , monnaie de départ  $debut$ , nombre max
   d'échanges  $p$ 
2: Sorties : ( $meilleur\_gain, meilleur\_chemin$ ) le cycle partant de  $debut$  avec  $\leq p$  échanges max-
   imisant les gains
3: procédure DFS( $noeud, gain, chemin, profondeur$ )
4:    $meilleur\_gain \leftarrow 0.0$ 
5:    $meilleur\_chemin \leftarrow []$ 
6:   if  $noeud = debut$  and  $profondeur > 0$  then
7:      $meilleur\_gain \leftarrow gain$ 
8:      $meilleur\_chemin \leftarrow chemin$ 
9:   if  $profondeur = p$  then
10:    return ( $meilleur\_gain, meilleur\_chemin$ )
11:   for each  $sommet, arete$  in  $G[noeud]$  do
12:     ( $gain', chemin'$ )  $\leftarrow$  DFS( $sommet, gain \times arete, chemin \cup [sommet], profondeur + 1$ )
13:     if  $gain' > meilleur\_gain$  then
14:        $meilleur\_gain \leftarrow gain'$ 
15:        $meilleur\_chemin \leftarrow chemin'$ 
16: return DFS( $debut, 1.0, [debut], 0$ )

```

Algorithm 2 meilleur_cycle_ameliore($G, debut, p$), temps: $O(p|U|)$, mémoire: $O(p|X|)$

```

1: Entrées : dictionnaire d'adjacence  $G$  des taux  $G[i, j]$ , monnaie de départ  $debut$ , nombre max
   d'échanges  $p$ 
2: Sorties : ( $meilleur\_gain, meilleur\_chemin$ ) le cycle partant de  $debut$  avec  $\leq p$  échanges max-
   imisant les gains
3:  $dp[k][sommet] \leftarrow 0.0$  pour tout  $k \in [0, p]$  et  $sommet \in G$ 
4:  $parent[k][sommet] \leftarrow \text{NULL}$  pour tout  $k \in [0, p]$  et  $sommet \in G$ 
5:  $dp[0][debut] \leftarrow 1.0$ 
6: for  $k \leftarrow 1$  to  $p$  do
7:    $flag \leftarrow \text{False}$ 
8:   for each  $sommet$  in  $G$  do
9:     if  $dp[k-1][sommet] \leq 0.0$  then
10:      Continue
11:      $gain \leftarrow dp[k-1][sommet]$ 
12:     for each  $sommet', arete$  in  $G[sommet]$  do
13:        $gain' \leftarrow gain \times arete$ 
14:       if  $gain' > dp[k][sommet']$  then
15:          $dp[k][sommet'] \leftarrow gain'$ 
16:          $parent[k][sommet'] \leftarrow sommet$ 
17:        $flag \leftarrow \text{True}$ 
18:   if not  $flag$  then
19:     break
20:  $meilleur\_gain \leftarrow 0.0$ 
21:  $meilleur\_k \leftarrow \text{NULL}$ 
22: for  $k \leftarrow 1$  to  $p$  do
23:   if  $dp[k][debut] > meilleur\_gain$  then
24:      $meilleur\_gain \leftarrow dp[k][debut]$ 
25:      $meilleur\_k \leftarrow k$ 
26: if  $meilleur\_k$  is  $\text{NULL}$  then
27:   return  $0.0 []$ 
28:  $cycle \leftarrow []$ 
29:  $monnaie \leftarrow debut$ 
30: for  $i \leftarrow meilleur\_k$  to  $1$  with step  $-1$  do
31:    $tmp \leftarrow parent[i][monnaie]$ 
32:    $cycle.append(tmp)$ 
33:    $monnaie \leftarrow tmp$ 
34:  $cycle.reverse()$ 
35:  $meilleur\_cycle \leftarrow cycle \cup [debut]$ 
36: return ( $meilleur\_gain, meilleur\_cycle$ )

```
